

Resolução — Exame Modelo 2 —

Paradigmas de Programação

Época de Recurso | Ano Letivo: 2025/2026

PARTE 1 – Perguntas Teóricas

Pergunta 1

As exceções em Java são objetos que representam condições anómalas ou situações de erro que ocorrem durante a execução de um programa. Todas as exceções derivam da classe Throwable, que possui duas subclasses principais: Error e Exception. Os Errors representam falhas graves do sistema que não devem ser tratadas pelo programador, como falta de memória. As Exceptions representam condições recuperáveis que o programador deve considerar e tratar.

As exceções verificadas (checked exceptions) são subclasses diretas de Exception que não estendem RuntimeException. O compilador obriga a que estas exceções sejam explicitamente tratadas pelo programador, quer através de um bloco try-catch, quer através da declaração throws na assinatura do método. São utilizadas para representar situações previsíveis e recuperáveis, como erros de entrada e saída ou ficheiros não encontrados. As exceções não verificadas (unchecked exceptions) são subclasses de RuntimeException. O compilador não obriga ao seu tratamento. São utilizadas para representar erros de programação, como acessos a índices inválidos de arrays ou divisões por zero.

O mecanismo try-catch-finally funciona da seguinte forma: o bloco try contém o código que pode gerar uma exceção. O bloco catch captura e trata exceções de tipos específicos. O bloco finally é opcional e contém código que é executado sempre, independentemente de ter ou não ocorrido uma exceção, sendo tipicamente utilizado para libertar recursos. É adequado criar exceções personalizadas quando se pretende representar erros específicos do domínio da aplicação que não são adequadamente representados pelas exceções predefinidas do Java.

```
public class RouteException extends Exception {
    public RouteException(String message) {
        super(message);
    }
}

public class GestorDeRotas {
    public void adicionarPonto(String ponto) throws RouteException {
        if (ponto == null || ponto.isEmpty()) {
            throw new RouteException("Ponto da rota invalido.");
        }
        System.out.println("Ponto adicionado: " + ponto);
    }

    public static void main(String[] args) {
        GestorDeRotas gestor = new GestorDeRotas();
        try {
            gestor.adicionarPonto("Zona A");
            gestor.adicionarPonto(null);
        } catch (RouteException e) {
            System.out.println("Erro capturado: " + e.getMessage());
        } finally {
            System.out.println("Operacao de rota concluida.");
        }
    }
}
```

Pergunta 2

Os construtores em Java são blocos especiais de código que são invocados automaticamente no momento da criação de um objeto através do operador new. O construtor tem obrigatoriamente o mesmo nome que a classe e não possui tipo de retorno, nem mesmo void. A sua função principal é inicializar o estado do objeto, atribuindo valores iniciais aos atributos de instância.

O construtor por defeito é um construtor sem parâmetros que o compilador Java gera automaticamente quando a classe não define explicitamente nenhum construtor. Este construtor por defeito invoca implicitamente o construtor sem parâmetros da superclasse (super()) e inicializa todos os atributos de instância com os seus valores padrão: zero para numéricos, false para

booleanos e null para referências. Quando a classe define explicitamente pelo menos um construtor, o compilador deixa de gerar o construtor por defeito.

O construtor parametrizado é aquele que recebe um ou mais argumentos, permitindo personalizar a inicialização do objeto. O encadeamento de construtores com `this` permite invocar outro construtor da mesma classe, evitando a duplicação de lógica de inicialização. O encadeamento com `super` permite invocar explicitamente um construtor da superclasse para inicializar a parte herdada do estado. Tanto `this` quanto `super` devem ser a primeira instrução do corpo do construtor e são mutuamente exclusivos.

```
public class Contentor {
    private String codigo;
    private double capacidade;
    private double pesoAtual;

    public Contentor(String codigo, double capacidade) {
        this(codigo, capacidade, 0);
    }

    public Contentor(String codigo, double capacidade, double pesoInicial) {
        this.codigo = codigo;
        this.capacidade = capacidade;
        this.pesoAtual = pesoInicial;
    }
}

public class ContentorRefrigerado extends Contentor {
    private double temperatura;

    public ContentorRefrigerado(String codigo, double capacidade, double temperatura) {
        super(codigo, capacidade);
        this.temperatura = temperatura;
    }
}
```

Pergunta 3

Os tipos enumerados (enum) em Java são um tipo especial de classe que representa um conjunto fixo e predefinido de constantes. Cada valor do enum é uma instância singleton da classe

enumerada, ou seja, existe uma e apenas uma instância para cada valor do enum durante toda a execução do programa.

A utilização de enum apresenta vantagens significativas em relação às constantes inteiras ou Strings. Em primeiro lugar, garante a segurança de tipos em tempo de compilação, impedindo que valores inválidos sejam atribuídos a variáveis do tipo enum. Se tentarmos atribuir um valor que não pertence ao enum, o compilador rejeita o código. Em segundo lugar, melhora a legibilidade do código porque os valores são referenciados por nomes descritivos. Em terceiro lugar, permite a utilização em instruções switch de forma natural. Em quarto lugar, a comparação entre valores de enum pode ser feita com segurança utilizando o operador de igualdade referencial, dispensando o uso de equals.

Um enum em Java pode possuir atributos privados, construtores privados e métodos, comportando-se como uma classe com capacidades adicionais. O construtor de um enum é obrigatoriamente privado porque as instâncias são criadas internamente pelo Java e não podem ser instanciadas externamente.

```
public enum ItemType {  
    PERISHABLE_FOOD("Alimentos Pereciveis", 1),  
    NON_PERISHABLE_FOOD("Alimentos Nao Pereciveis", 2),  
    CLOTHING("Vestuario", 3),  
    MEDICINE("Medicamentos", 4);  
  
    private String descricao;  
    private int prioridade;  
  
    ItemType(String descricao, int prioridade) {  
        this.descricao = descricao;  
        this.prioridade = prioridade;  
    }  
  
    public String getDescricao() {  
        return descricao;  
    }  
  
    public int getPrioridade() {  
        return prioridade;  
    }  
}
```

Pergunta 4

A composição e a herança são dois mecanismos de reutilização de código em Java com propósitos distintos. A herança estabelece uma relação hierárquica entre classes onde a subclasse herda os membros da superclasse, representando uma relação do tipo é-um. A composição estabelece uma relação em que um objeto contém outro objeto como atributo, representando uma relação do tipo tem-um.

A composição é frequentemente preferida em relação à herança por várias razões. Em primeiro lugar, permite maior flexibilidade na composição de comportamentos, pois os objetos contidos podem ser alterados em tempo de execução. Em segundo lugar, reduz o acoplamento entre classes porque a classe composta não depende da implementação interna da classe contida, mas apenas da sua interface pública. Em terceiro lugar, evita a fragilidade da herança, onde alterações na superclasse podem impactar inesperadamente as subclasses.

```
public class Motor {  
    private int potencia;  
  
    public Motor(int potencia) {  
        this.potencia = potencia;  
    }  
  
    public void ligar() {  
        System.out.println("Motor de " + potencia + "cv ligado.");  
    }  
  
    public int getPotencia() {  
        return potencia;  
    }  
}  
  
public class Veiculo {  
    private String matricula;  
    private Motor motor;  
  
    public Veiculo(String matricula, Motor motor) {  
        this.matricula = matricula;  
        this.motor = motor;  
    }  
  
    public void arrancar() {  
        motor.ligar();  
        System.out.println("Veiculo " + matricula + " em movimento.");  
    }  
}
```

PARTE 2 – Programação em Java

Pergunta 1a

```
public class RouteImpl implements Route {  
    private static final int MAX_AIDBOXES = 10;  
    private Vehicle vehicle;  
    private AidBox[] aidBoxes;  
    private int numberOfAidBoxes;  
  
    public RouteImpl(Vehicle vehicle) {  
        if (vehicle == null) {  
            throw new IllegalArgumentException("O veiculo nao pode ser nulo.");  
        }  
        this.vehicle = vehicle;  
        this.aidBoxes = new AidBox[MAX_AIDBOXES];  
        this.numberOfAidBoxes = 0;  
    }  
  
    @Override  
    public Vehicle getVehicle() {  
        return this.vehicle;  
    }  
  
    @Override  
    public void addAidBox(AidBox aidBox) throws RouteException {  
        if (aidBox == null) {  
            throw new RouteException("A AidBox nao pode ser nula.");  
        }  
        if (numberOfAidBoxes >= MAX_AIDBOXES) {  
            throw new RouteException("Capacidade maxima da rota atingida.");  
        }  
        for (int i = 0; i < numberOfAidBoxes; i++) {  
            if (aidBoxes[i].equals(aidBox)) {  
                throw new RouteException("A AidBox ja existe na rota.");  
            }  
        }  
        aidBoxes[numberOfAidBoxes] = aidBox;  
        numberOfAidBoxes++;  
    }  
}
```

```
@Override
public AidBox removeAidBox(AidBox aidBox) throws RouteException {
    if (aidBox == null) {
        throw new RouteException("A AidBox nao pode ser nula.");
    }
    for (int i = 0; i < numberOfAidBoxes; i++) {
        if (aidBoxes[i].equals(aidBox)) {
            AidBox removed = aidBoxes[i];
            for (int j = i; j < numberOfAidBoxes - 1; j++) {
                aidBoxes[j] = aidBoxes[j + 1];
            }
            aidBoxes[numberOfAidBoxes - 1] = null;
            numberOfAidBoxes--;
            return removed;
        }
    }
    throw new RouteException("AidBox nao encontrada na rota.");
}
```

```
@Override
public AidBox[] getRoute() {
    AidBox[] result = new AidBox[numberOfAidBoxes];
    for (int i = 0; i < numberOfAidBoxes; i++) {
        result[i] = aidBoxes[i];
    }
    return result;
}
```

```
@Override
public boolean equals(Object obj) {
    if (this == obj) {
        return true;
    }
    if (obj == null) {
        return false;
    }
    if (!(obj instanceof Route)) {
        return false;
    }
    Route other = (Route) obj;
```

```
        return this.vehicle.getCode().equals(other.getVehicle().getCode());
    }

    @Override
    public String toString() {
        return "Rota [Veiculo: " + vehicle.getCode() + " | AidBoxes: " + numberOfAidBoxes + "];"
    }
}
```

Pergunta 1b

```
public class TestRoute {  
    public static void main(String[] args) {  
        VehicleImpl v1 = new VehicleImpl("VH-001", ItemType.PERISHABLE_FOOD, 500.0);  
        RouteImpl r1 = new RouteImpl(v1);  
  
        System.out.println("Veiculo da rota: " + r1.getVehicle().getCode());  
  
        AidBoxImpl ab1 = new AidBoxImpl("AB-001", "Zona Norte");  
        AidBoxImpl ab2 = new AidBoxImpl("AB-002", "Zona Sul");  
  
        try {  
            r1.addAidBox(ab1);  
            System.out.println("AidBox AB-001 adicionada com sucesso.");  
            r1.addAidBox(ab2);  
            System.out.println("AidBox AB-002 adicionada com sucesso.");  
        } catch (RouteException e) {  
            System.out.println("Erro: " + e.getMessage());  
        }  
  
        System.out.println("Numero de AidBoxes na rota: " + r1.getRoute().length);  
  
        try {  
            r1.addAidBox(ab1);  
        } catch (RouteException e) {  
            System.out.println("Erro esperado (duplicada): " + e.getMessage());  
        }  
  
        try {  
            AidBox removida = r1.removeAidBox(ab1);  
            System.out.println("AidBox removida: " + removida.getCode());  
        } catch (RouteException e) {  
            System.out.println("Erro: " + e.getMessage());  
        }  
  
        System.out.println("Numero de AidBoxes apos remocao: " + r1.getRoute().length);  
  
        RouteImpl r2 = new RouteImpl(v1);  
        System.out.println("Igualdade r1 com r2 (mesmo veiculo): " + r1.equals(r2));  
        System.out.println("Representacao textual: " + r1.toString());  
    }  
}
```

```
}  
}
```

Pergunta 2a

```
public class CollectionManagerImpl implements CollectionManager {  
  
    private double getContainerLoad(Container container) {  
        if (container == null) {  
            return 0;  
        }  
        Measurement last = container.getLastMeasurement();  
        if (last == null) {  
            return 0;  
        }  
        return last.getValue();  
    }  
  
    private boolean isContainerFull(Container container, double threshold) {  
        if (container == null) {  
            return false;  
        }  
        Measurement last = container.getLastMeasurement();  
        if (last == null) {  
            return false;  
        }  
        return last.getValue() > (container.getCapacity() * threshold / 100);  
    }  
  
    @Override  
    public double getTotalCollectedByType(IIInstitution inst, ItemType type) {  
        return 0;  
    }  
}
```

Pergunta 2b

```
public class CollectionManagerImpl implements CollectionManager {

    private double getContainerLoad(Container container) {

        if (container == null) {
            return 0;
        }

        Measurement last = container.getLastMeasurement();
        if (last == null) {
            return 0;
        }
        return last.getValue();
    }

    private boolean isContainerFull(Container container, double threshold) {

        if (container == null) {
            return false;
        }

        Measurement last = container.getLastMeasurement();
        if (last == null) {
            return false;
        }
        return last.getValue() > (container.getCapacity() * threshold / 100);
    }

    @Override
    public double getTotalCollectedByType(IIInstitution inst, ItemType type) {

        if (inst == null || type == null) {
            return 0;
        }

        AidBox[] aidBoxes = inst.getAidBoxes();
        if (aidBoxes == null) {
            return 0;
        }

        double totalCarga = 0;
        for (int i = 0; i < aidBoxes.length; i++) {
            if (aidBoxes[i] == null) {
                continue;
            }
            Container[] containers = aidBoxes[i].getContainers();
```

```
    if (containers == null) {  
        continue;  
    }  
    for (int j = 0; j < containers.length; j++) {  
        if (containers[j] == null) {  
            continue;  
        }  
        if (containers[j].getType() == type) {  
            if (isContainerFull(containers[j], 75)) {  
                totalCarga += getContainerLoad(containers[j]);  
            }  
        }  
    }  
    }  
    return totalCarga;  
}  
}
```